T2 — HTML5 semántico, estructura de documento y accesibilidad base

1. Construir una **estructura HTML5 semántica** y justificarla (landmarks, jerarquía, componentes).
2. Explicar el **modelo de accesibilidad base** (roles, labels, foco, navegación por teclado) aplicado a HTML.
3. Diferenciar **HTML estático vs HTML dinámico** y decidir en qué casos conviene cada enfoque.
4. Aplicar criterios de **validación y calidad** (parser/validator, errores típicos, checklist de revisión de index.html).

Qué problema resuelve HTML

- HTML define **estructura** y **significado** del contenido
- CSS define presentación
- JS define comportamiento
- Separación de responsabilidades = mantenibilidad + calidad

Ejemplo: un `<button>` no es "un rectángulo": es un **control interactivo** con semántica y eventos.

HTML5 y el cambio de paradigma

HTML5 introduce:

- Etiquetas semánticas (header, nav, main, article, section, aside, footer)
- Media nativo (`audio`, `video`, `track`)
- Formularios modernos (types, validación)
- APIs nativas (pero se ven en sesiones posteriores)

Conexión con evaluación: se revisa la estructura HTML5 semántica y la accesibilidad

Semántica: definición operativa

Semántica = que el marcado exprese el rol real del contenido.

- "Esto es un título" → `<h1>`
- "Esto es navegación" → `<nav>`
- "Esto es contenido principal" → `<main>`
- "Esto es una pieza independiente" → `<article>`

Nota: la semántica mejora accesibilidad, SEO y mantenibilidad.

Documento HTML mínimo correcto

<!doctype html>

<html lang="...">

<meta charset="utf-8">

<meta name="viewport" content="width=device-width, initial-scale=1">

Ingeniería: estos 4 elementos evitan problemas sistemáticos (render, i18n, responsive).

Plantilla base para el curso

```
<!doctype html>

<html lang="es">

<head>

  <meta charset="utf-8">

  <meta name="viewport" content="width=device-width, initial-scale=1">



  <title>Mi Web-App</title>

  <meta name="description" content="Descripción breve y precisa.">



  <!-- Recomendados -->

  <meta name="theme-color" content="#ffffff">

  <link rel="icon" href="/favicon.ico">

  <!-- <link rel="canonical" href="https://tudominio.com/"> -->



  <!-- CSS -->

  <link rel="stylesheet" href="/assets/css/styles.css">



  <!-- JS (no bloquea render) -->

  <script type="module" src="/assets/js/app.js" defer></script>

</head>



<body>
```

```
  <a class="skip-link" href="#main">Saltar al contenido</a>


  <header>

    <!-- Recomendado: branding + navegación -->

    <nav aria-label="Navegación principal">

      <!-- ... -->

    </nav>

  </header>


  <main id="main" tabindex="-1">

    <h1>Título de la página</h1>

    <!-- ... -->

  </main>


  <footer>

    <!-- ... -->

  </footer>

</body>

</html>
```

<a class="skip-link" href="#main">Saltar al contenido</a> es un enlace de accesibilidad (skip link) que permite a quien navega con teclado o lector de pantalla saltarse bloques repetidos (cabecera, menú) e ir directamente al contenido principal.

Encoding e internacionalización (por qué importa)

- El navegador interpreta bytes → caracteres según encoding
- `utf-8` es estándar para compatibilidad global
- lang="ca" / "es" / "en" afecta:
  - pronunciación en lectores de pantalla
  - hyphenation / correctores
  - indexación

Conexión con guía: internacionalización/encoding se trata como concepto base de formatos y validación

Landmarks: el "mapa" de la página

Landmarks principales:

- `header` (cabecera de bloque)
- `nav` (navegación)
- `main` (contenido principal, único)
- `aside` (complementario)
- `footer` (pie)
- `section` (agrupa por tema)
- `article` (unidad independiente)

Regla práctica: si puedes "entender" la página solo mirando etiquetas, vas bien.

`header` y `footer` no son "arriba/abajo"

- `header` puede aparecer dentro de un `article`
- `footer` puede aparecer dentro de una `section`

Ejemplo:

- `header` del sitio (logo+nav)
- `header` de un artículo (título+autor+fecha)

`main`: reglas estrictas

- Debe existir **uno** por documento
- No puede ser descendiente de `header`, `nav`, `footer`, `article`, `aside`, `section`
- Debe contener lo "principal" para esa vista

Motivo: accesibilidad (saltos directos al contenido).

`nav`: navegación con nombre accesible

- Puede haber múltiples `nav` (principal, breadcrumbs, footer links)
- Deben distinguirse con `aria-label`

Ejemplo:

- aria-label="Navegación principal"
- aria-label="Navegación secundaria"

aria-label es un **atributo ARIA** que sirve para darle a un elemento un **nombre accesible** (una "etiqueta") que los **lectores de pantalla** anuncian.

En otras palabras: **no cambia el aspecto visual**, pero sí cambia **cómo se identifica** ese elemento para accesibilidad.

`section` vs `article` (confusión típica)

- `article`: contenido autónomo reutilizable (post, ficha evento, noticia)
- `section`: agrupación temática dentro de un documento

Test de ingeniería: ¿podría "vivir solo" y seguir teniendo sentido? → `article`.

Jerarquía de headings: el backbone del documento

- `h1` = título de la vista/página
- `h2` = secciones principales
- `h3` = subsecciones…

Reglas:

- No "saltos" por estética (no pasar de h2 a h4)
- No usar headings para tamaño; eso es CSS

Outline (modelo conceptual)

Una página bien estructurada es navegable por:

- teclado (Tab / Shift+Tab)
- lectores de pantalla (saltos por heading)
- motores de búsqueda (estructura)

Consecuencia: headings mal usados = accesibilidad mala + SEO peor.

Contenido textual: `p`, `ul/ol`, `dl`

- Texto normal: `p`
- Listas: `ul`, `ol` (siempre que sea "colección de items")
- Definiciones/diccionario: `dl` con `dt/dd`

Ejemplo

<dl> se usa para marcar una lista de pares "término → definición" (o "nombre → valor"). Es ideal para glosarios, FAQs tipo concepto/explicación, fichas técnicas, o metadatos.

Estructura

- <dl>: contenedor de la lista.
- <dt>: *definition term* (el término / etiqueta).
- <dd>: *definition description* (la definición / valor).

Ejemplo básico

```
<dl>
```

```
  <dt>Bitrate</dt>

  <dd>Cantidad de bits por segundo usados para codificar un flujo de audio o
vídeo.</dd>



  <dt>Códec</dt>

  <dd>Algoritmo para comprimir y descomprimir datos multimedia (p. ej., H.264,
AV1).</dd>

</dl>
```

Imágenes: `img` + `alt` (regla de oro)

- `alt` describe la **función** de la imagen, no su estética
- Si es decorativa → `alt=""` (vacío)
- Si aporta información → alt descriptivo y específico

Evaluación: textos alternativos y accesibilidad se revisan explícitamente

Logo de marca

<img src="logo-bindi.svg" alt="Bindi Yoga Lab">

Imagen que describe una acción

<a href="/contacto">

  <img src="icono-whatsapp.svg" alt="Contactar por WhatsApp">

</a>

figure y figcaption

Para contenido visual con contexto:

<figure>

  <img src="..." alt="Cartel del festival 2026">

  <figcaption>Festival X — edición 2026, Palma</figcaption>

</figure>

Beneficio: asociación semántica explícita.

Enlaces: accesibles y útiles

Errores típicos:

- “Haz click aquí” (sin contexto)
- Link en icono sin `aria-label`

Buenas prácticas:

- Texto de link descriptivo (“Ver programación completa”)
- Si es icono: aria-label="Abrir mapa"

Botones vs enlaces (decisión de ingeniería)

- `<a>` navega / cambia URL / referencia recurso
- `<button>` ejecuta acción en la misma vista (filtrar, abrir modal)

Antipatrón: usar `<div>` clickable (rompe teclado/semántica).

Formularios: estructura y labels

Requisitos:

- `label` asociado a `input` (for/id)
- `name` coherente para datos
- type correcto (email, date, search)

Evaluación: Forms API + validación (más adelante), pero la base es HTML correcto

Formularios: ejemplo mínimo correcto

```
<form action="/buscar" method="get">

  <label for="q">Buscar</label>

  <input id="q" name="q" type="search" autocomplete="off">

  <button type="submit">Buscar</button>

</form>
```

1) <label for="q">Buscar</label>

Crea una etiqueta textual para el campo.

for="q" enlaza el label con el input cuyo id="q".

Beneficios:

Accesibilidad: el lector de pantalla anuncia “Buscar, campo de búsqueda”.

Usabilidad: al hacer clic en “Buscar”, se activa el campo.

2) <input id="q" name="q" type="search" autocomplete="off">

id="q": identificador único en la página. Debe coincidir con el for del label.

name="q": nombre del parámetro que se enviará al servidor/URL (por ejemplo ?q=texto).

type="search": indica que es un campo de búsqueda.

En móvil suele mostrar teclado con botón “Buscar”.

Algunos navegadores añaden un icono para limpiar (×).

autocomplete="off": intenta impedir que el navegador sugiera búsquedas anteriores.

Ojo: algunos navegadores pueden ignorarlo en ciertos contextos.

3) <button type="submit">Buscar</button>

Botón que envía el formulario (si está dentro de un <form>).

type="submit":

Si está dentro de un form, dispara el envío.

Si no hay form, el botón no hará “submit” de nada (no tiene efecto por sí solo).

Accesibilidad (A11y): modelo simplificado

Pilares:

1. Perceptible (texto alternativo, contraste)
2. Operable (teclado, foco visible)
3. Comprensible (labels, mensajes)
4. Robusto (semántica correcta)

Conexión con curso: accesibilidad es criterio explícito de evaluación

Navegación por teclado: lo mínimo exigible

- Todo lo interactivo debe ser alcanzable con Tab
- Orden de tabulación coherente
- Foco visible (no eliminar outline sin alternativa)
- Evitar “trampas de foco” (modales mal hechos)

ARIA: cuándo sí y cuándo no

Regla de oro:

- Primero HTML nativo.
- ARIA solo si no hay elemento semántico equivalente.

Ejemplos:

- Correcto: nav aria-label="..."
- Incorrecto: `div role="button"` existiendo `<button>`

Evaluación: ARIA y accesibilidad están en checklist

HTML estático vs HTML dinámico (definiciones)

HTML estático

- El servidor entrega un archivo `.html` "cerrado"
- Contenido cambia solo si regeneras/subes archivos

HTML dinámico

- Contenido se genera a partir de datos (en cliente o servidor)
- La vista es resultado de: datos + plantilla + lógica

Conexión con evaluación: se revisa "HTMLs estáticos vs dinámicos" y `index.html`

Dinámico en cliente vs dinámico en servidor

Server-side rendering (SSR)

- HTML se genera en servidor → llega listo
- Ventajas: SEO inicial, performance percibida
- Coste: lógica en servidor

Client-side rendering (CSR)

- HTML base + JS que renderiza desde JSON
- Ventajas: interactividad, SPA
- Coste: depende de JS, SEO requiere estrategia

Nota: en este curso, el enfoque típico será CSR + JSON, sin impedir SSR conceptual.

Caso real: catálogo "data-driven"

Objetivo: no escribir 50 tarjetas a mano.

- Guardas datos en `dataset.json`
- Renderizas tarjetas dinámicamente
- Filtras por categoría/fecha/isla

Uso para P2 y P3: Web-App basada en datos (guía de prácticas)

Anti-patrones comunes en HTML de alumnos

1. IDs duplicados
2. Headings usados por tamaño
3. Botones con `<div>`
4. Formularios sin `label`
5. `alt` inexistente o inútil
6. `main` inexistente
7. `lang` ausente
8. Estructura basada en `<br>` y no en bloques

Validación: parser vs validator (concepto ingeniería)

- **Parser:** interpreta HTML y construye el DOM (siempre "hace lo que puede")
- **Validator:** compara tu documento con reglas del estándar y reporta errores

Parser (navegador)

- Qué es: el componente del navegador que lee el HTML, lo interpreta y construye el DOM (Document Object Model).
- Objetivo de ingeniería: robustez. El parser está diseñado para "hacer lo que pueda" aunque el HTML tenga errores, porque en la web real hay muchísimo marcado imperfecto.
- Qué hace cuando hay errores: aplica error-recovery (recuperación):
  - "cierra" etiquetas que faltan,
  - "inserta" elementos implícitos,
  - reordena nodos para que el árbol DOM sea válido internamente,
  - intenta asegurar que la página se renderice (se vea).

Validator (herramienta de validación)

- Qué es: una herramienta que compara tu documento con las reglas del estándar (HTML, accesibilidad, etc.) y reporta errores/warnings.
- Objetivo de ingeniería: conformidad (compliance) y calidad:
  - detectar errores de estructura/atributos,
  - asegurar semántica correcta,
  - evitar comportamientos inesperados entre navegadores,
  - mejorar mantenibilidad, SEO y accesibilidad.
    En la evaluación se insiste en revisar estructura/semántica y accesibilidad, y en usar herramientas de evaluación como WAVE/Axe/Lighthouse.

Diferencia clave

- Parser: "*lo renderizo como pueda*" → produce un DOM (aunque no sea el que tú creías).
- Validator: "*esto cumple / esto NO cumple el estándar*" → produce un informe de errores.

Idea clave: que el navegador "lo muestre" no significa que sea correcto.

Ejemplo típico (por qué importa)

Caso A: etiquetas mal anidadas o sin cerrar

Si olvidas un cierre o anidas mal, el navegador puede auto-cerrar o mover elementos.

- Resultado: el DOM final no coincide con tu intención.
- Síntoma: CSS/JS "no funciona" como esperabas (selectores que no encajan, eventos que no disparan donde toca).

Caso B: IDs duplicados

<main id="main">...</main>

<section id="main">...</section>

- El navegador lo mostrará, pero:
  - getElementById("main") puede devolverte un elemento distinto al esperado.
  - un skip-link href="#main" puede saltar a un sitio incorrecto.
- El validator lo marcaría como error (porque un id debe ser único).

Validación HTML: qué se busca realmente

- Semántica coherente
- Estructura sin errores graves
- Atributos correctos
- Accesibilidad base (complementariamente con herramientas)

Evaluación: "HTML semánticamente correcto", "bien estructurado y accesible"

index.html como "contrato" del proyecto

- Es la puerta de entrada: estructura + navegación + landmarks
- Si `index.html` está bien, el resto del proyecto tiende a estar bien

Evaluación: revisión específica de `index.html`

HTML para componentes: patrón mental

Componentes típicos en una Web-App:

- **Card** (evento/obra/lugar)
- **List view** (resultados)
- Detail view (ficha)
- **Filters** (form)
- **Navigation** (tabs, menu)

Regla: cada componente debe tener semántica adecuada (article, headings, lists, buttons).

Ejemplo: Card semántica (evento)

```
<article class="card">

  <h2>Concierto de cámara</h2>

  <p><time datetime="2026-03-14">14 Mar 2026</time> · Palma</p>

  <p>Descripción breve...</p>

  <a href="event.html?id=123">Ver detalles</a>

</article>
```

Nota: `article` + `time` + link descriptivo.

ideas clave

- HTML5 es un **lenguaje de modelado** del contenido
- Semántica = accesibilidad + SEO + código mantenible
- Estático vs dinámico es una decisión de arquitectura
- Validación es parte del proceso de ingeniería (no un extra)